

add collision for the simulation

Query successful

Gemini said

To add collision detection and response to your simulation without modifying Particle.cpp or Particle.hpp, we can implement a "Spring-Dashpot" or "Elastic Impact" model directly within the gravityModel function (or a new dedicated function) in NBodyProblem.cpp.

Since the Particle class uses Euler integration (updating velocity then position), the simplest way to handle collisions is to check the distance between particles and, if they overlap, manually adjust their velocities based on the conservation of momentum.

Updated gravityModel in NBodyProblem.cpp

Replace your existing gravityModel function with the version below. This version calculates gravity as usual, but if it detects that two particles are touching (distance $\leq$ sum of radii), it applies a simple elastic collision impulse.

C++

```
void gravityModel(std::vector<Particle> &p) {  
    for (std::size_t idx = 0; idx < p.size(); idx++) {  
  
        // reset acceleration vector  
        Eigen::Vector2f total_accel{0.0, 0.0};  
  
        for (std::size_t jdx = 0; jdx < p.size(); jdx++) {
```

```

if (idx == jdx) continue;

// Calculate relative position and distance
Eigen::Vector2f r_ji = p[idx].position - p[jdx].position;
float r_mag = r_ji.norm();

// --- 1. Collision Handling (Elastic Impact) ---
float min_dist = p[idx].radius + p[jdx].radius;

if (r_mag < min_dist) {
    // To prevent sticking, we only resolve if they are moving toward each other
    Eigen::Vector2f relative_vel = p[idx].velocity - p[jdx].velocity;
    float velocity_along_normal = relative_vel.dot(r_ji.normalized());

    if (velocity_along_normal < 0) {
        // Simple perfectly elastic collision formula for velocity change
        // This uses a simplified impulse based on equal masses (since mass=10 for all)
        // If you want varying masses: impulse = (2 * v_rel) / (m1 + m2)
        float restitution = 0.8f; // 1.0 = bouncy, 0.5 = sluggish
        float impulse_mag = -(1.0f + restitution) * velocity_along_normal;
        impulse_mag /= (1.0f / p[idx].mass + 1.0f / p[jdx].mass);

        Eigen::Vector2f impulse = impulse_mag * r_ji.normalized();

        // Apply velocity change immediately to prevent overlapping "stickiness"
        p[idx].velocity += (1.0f / p[idx].mass) * impulse;
    }
}

```

```

    }

    // Skip gravity if they are colliding to avoid singularities
    continue;
}

// --- 2. Standard Gravity Model ---

// Calculate gravitational acceleration
total_accel -= 1.0 * G * p[jdx].mass / std::pow(r_mag, 3) * r_ji;
}

p[idx].accel = total_accel;
}
}

```

Key Changes Explained:

Distance Check: We compare `r_mag` against `min_dist` (`radiusi+radiusj`). Your previous code used `2.0 * p[idx].radius`, which only worked if all particles were the exact same size.

Relative Velocity: We check `velocity_along_normal`. This ensures that particles only "bounce" if they are moving toward each other. This prevents them from getting stuck inside one another if the integration step is too large.

Impulse Calculation: Instead of just flipping velocities, we calculate an impulse based on the normal vector between the two centers. I've added a restitution variable (0.8) so the collisions feel a bit more natural (losing a tiny bit of energy).

Immediate Velocity Update: Because gravityModel is called before p.update(dt), modifying the velocity here ensures the next position calculation reflects the "bounce."

Optional: Static Overlap Correction

If your particles still get stuck together during high-speed clusters, you can add this line inside the if (r_mag < min_dist) block to physically push them apart:

C++

```
// Prevent overlapping (Static Resolution)
```

```
float overlap = 0.5f * (min_dist - r_mag);
```

```
p[idx].position += (r_ji / r_mag) * overlap;
```

```
p[jdx].position -= (r_ji / r_mag) * overlap;
```

Would you like me to help you implement a more advanced integration method like RK4 to improve the stability of these collisions?

Gemini is AI and can make mistakes.